Plantilla de Análisis Principios SOLID

Proyecto: Sistema de Gestión de Préstamo de Recursos de Laboratorio **Integrante:** Victor Alejandro Aristizabal Ospina **Docente:** Alejandra Guerrero Bocanegra **Curso:** Diseño de Sistemas de Información — ITM **Entregable:** Insumo para el Entregable No. 2 (Release 1 — MVP) **Fecha:** 14/10/2026

> Instrumento de apoyo para justificar los principios SOLID dentro del alcance de un producto mínimo viable. El análisis se realiza sobre el **diseño técnico** documentado en `.kiro/specs/frontend-sistema-prestamos-laboratorio/design.md`, dado que la implementación del MVP (tareas 1–22 de `tasks.md`) se encuentra en curso. Las referencias de evidencia usan el formato `design.md:Lx`.

---

1. Información general

Campo	Valor
Lenguaje / Framework	TypeScript (strict) + React 18/19 + Vite. SPA sin backend real; persistencia en `localStorage`.
Tipo de análisis	Análisis de **diseño** (arquitectura y contratos) con verificación de interfaces; validación posterior sobre el código cuando el MVP esté implementado.
Contexto	Proyecto de aula, alcance **MVP (Release 1)**. Cuatro capas: Pages → Features → Infraestructura (contexts/mockDb/router) → Shared. Estado con React Context + `useReducer` sincronizado con `localStorage` (`design.md:19-35` , `design.md:265-314`).
Fuente de evidencia	`.kiro/specs/frontend-sistema-prestamos-laboratorio/design.md` (HLD) y `tasks.md` (plan de implementación).

Componentes seleccionados (cubren las cuatro capas)

ID	Componente	Capa	Motivo de selección
C1	`services/mockDb/loans.service.ts`	Servicios	Núcleo del dominio (ciclo de préstamo); concentra reglas + persistencia + efectos sobre recursos.
C2	`services/mockDb/resources.service.ts`	Servicios	CRUD de inventario + máquina de estados de recurso.
C3	`services/mockDb/auth.service.ts`	Servicios	Autenticación, hash y sesión; caso de estudio de DIP.
C4	`contexts/AppDataContext.tsx`	Estado / Infraestructura	Estado global de dominio y persistencia (`useReducer` + sync).
C5	`contexts/AuthContext.tsx`	Estado	Estado de sesión global.
C6	`shared/utils/validators.ts` + `shared/utils/dateUtils.ts`	Shared	Funciones puras: contraste de "buen diseño" (SRP/OCP altos).
C7	`shared/components/guards/{PrivateRoute,RoleGuard}.tsx` + `shared/components/ui/StatusBadge.tsx`	Vistas	Contraste de "buen diseño" en la capa de presentación (OCP/ISP).

---

2. Objetivo del análisis

Analizar la aplicación de los principios **SRP, OCP, LSP, ISP y DIP** en los componentes que conforman el **MVP** del Sistema de Préstamos de Recursos de Laboratorio, con el fin de:

1. Identificar **fortalezas** de diseño ya presentes y que conviene preservar.
2. Detectar **problemas de diseño** (acoplamiento, responsabilidades mezcladas, puntos de extensión rígidos) antes de escribir el código del MVP.
3. Producir **recomendaciones de refactorización** priorizadas por **severidad, esfuerzo y relevancia**, que se incorporarán durante la implementación de las tareas 1–22 y se documentarán en el **Entregable No. 2**.

Alcance de la revisión: módulos de servicios (`mockDb`), contextos globales y utilidades/componentes compartidos. **Fuera de alcance:** funcionalidades Post-MVP (Release 2), renderizado visual detallado y pruebas de rendimiento.

---

3. Descripción breve de los componentes

C1 — `loans.service.ts`

- **Función principal:** gestionar el ciclo de vida del préstamo (reserva → entrega → devolución) y verificar disponibilidad.
- **Responsabilidades actuales:** validación de tiempos y motivo, verificación de solapamiento, creación/consulta de préstamos, cálculo de retraso, orquestación del cambio de estado del recurso y persistencia.
- **Clases / interfaces involucradas:** `Loan`, `LoanFormData`, `OperationResult<T>`, `LoanStatus`, `ResourceCondition` (`design.md:185-249`).
- **Dependencias principales:** `resources.service` (cambio de estado y cantidad), `dateUtils` (`intervalsOverlap`, `calculateDelayHours`), `storage/localStorage` (`design.md:378-420`).
- **Relación con otros módulos:** consumido por `features/loans` (`useLoans`, `useAvailability`) y `features/operations` (`useDeliveries`, `useReturns`) (`design.md:489-498`).

C2 — `resources.service.ts`

- **Función principal:** CRUD del catálogo de recursos y control de transiciones de estado.
- **Responsabilidades actuales:** crear/editar/listar/buscar recursos, dar de baja (con validación de préstamos activos) y validar transiciones de la máquina de estados registrando historial.
- **Clases / interfaces involucradas:** `Resource`, `ResourceStatus`, `ResourceStatusHistory`, `OperationResult<T>` (`design.md:152-182`, `design.md:245-249`).
- **Dependencias principales:** `loans`/consulta de préstamos activos, `storage/localStorage` (`design.md:397-403`, `design.md:420`).
- **Relación con otros módulos:** consumido por `features/inventory` (`useInventory`) y por `loans.service` para actualizar estado/cantidad.

C3 — `auth.service.ts`

- **Función principal:** registro, login, logout y lectura de sesión.
- **Responsabilidades actuales:** validar correo único y contraseña, hashear contraseña, generar `Session` con expiración, emitir error genérico en fallo.
- **Clases / interfaces involucradas:** `User`, `Session`, `OperationResult<T>` (`design.md:139-149`, `design.md:227-234`, `design.md:245-249`).
- **Dependencias principales:** `hashUtils`, `uuidUtils`, `storage/localStorage` (`design.md:392-395`).
- **Relación con otros módulos:** consumido por `AuthContext` y `features/auth` (`design.md:424-450`).

C4 — `AppDataContext.tsx`

- **Función principal:** exponer el estado de dominio (`resources`, `loans`, `users`, `sanctions`) y persistirlo.
- **Responsabilidades actuales:** cargar estado inicial desde `localStorage`, ejecutar `seedData()` si no está inicializado, reducir acciones y sincronizar con `localStorage` en cada cambio, manejar error si `localStorage` no está disponible.
- **Clases / interfaces involucradas:** `Resource`, `Loan`, `User`, `Sanction` (`design.md:92-98`, `design.md:134-225`).
- **Dependencias principales:** `services/mockDb`, `seed-data`, `localStorage` (`design.md:287-314`).
- **Relación con otros módulos:** base de los hooks de feature (`useInventory`, `useLoans`, `useDeliveries`, `useReturns`).

C5 — `AuthContext.tsx`

- **Función principal:** mantener la sesión activa de la aplicación.
- **Responsabilidades actuales:** `login`, `logout`, `register`; restaurar sesión al montar y validar expiración.
- **Clases / interfaces involucradas:** `Session`, `User` (`design.md:227-234`).
- **Dependencias principales:** `auth.service`, `localStorage` (`design.md:86-91`, `design.md:424-450`).
- **Relación con otros módulos:** consumido por guardias (`PrivateRoute`, `RoleGuard`) y por `AppShell` / `Sidebar` (`design.md:353-368`).

C6 — `validators.ts` + `dateUtils.ts`

- **Función principal:** validaciones puras de entrada y cálculos de fecha/hora.
- **Responsabilidades actuales:** `validatePassword`, `passwordsMatch`, `validateEmail`, `validateLoanTimes`, `validateMotivo`, `validateResourceName`, `validateQuantity`, `validateObservaciones`; `formatDate`, `formatDateTime`, `calculateDelayHours`, `isDateInPast`, `intervalsOverlap`.
- **Clases / interfaces involucradas:** funciones exportadas individualmente (sin estado).
- **Dependencias principales:** ninguna (funciones puras), salvo el tipo nativo `Date` (`tasks.md:27-48`).
- **Relación con otros módulos:** consumido por formularios, servicios y tests PBT (`design.md:599-701`).

C7 — `guards/{PrivateRoute,RoleGuard}.tsx` + `ui/StatusBadge.tsx`

- **Función principal:** control de acceso declarativo y presentación del estado del recurso/préstamo.
- **Responsabilidades actuales:** `PrivateRoute` redirige a `/login` sin sesión; `RoleGuard` redirige a `/access-denied` si el rol no está permitido; `StatusBadge` mapea estado → color/etiqueta.
- **Clases / interfaces involucradas:** `ResourceStatus`, `LoanStatus`, `UserRole` (`design.md:119-128`, `design.md:353-368`; `tasks.md:136-139`).
- **Dependencias principales:** `useAuth()` (abstracción de sesión) y React Router (`Navigate`).
- **Relación con otros módulos:** montados en `router.tsx` y en páginas; `StatusBadge` usado por tablas y tarjetas.

---

4. Resumen ejecutivo

Matriz de cumplimiento por componente

Leyenda: **Alto** = cumple · **Medio** = cumple parcialmente · **Bajo** = no cumple · **N/A** = no aplica.

Principio	C1 loans	C2 resources	C3 auth	C4 AppData	C5 Auth	C6 utils	C7 guards/UI
S — Single Responsibility	Bajo	Bajo	Medio	Bajo	Medio	Alto	Alto
O — Open/Closed	Bajo	Bajo	Medio	Medio	Medio	Alto	Alto
L — Liskov Substitution	Alto	Alto	Alto	Alto	Alto	Alto	N/A
I — Interface Segregation	Alto	Alto	Alto	Bajo	Medio	Alto	Alto
D — Dependency Inversion	Bajo	Bajo	Bajo	Bajo	Medio	Alto	Medio

Observación general

El diseño acierta en **separación por capas y por dominio**, y en los artefactos puros/declarativos (`utils`, `guards`). La **deuda de diseño** se concentra en la **capa de servicios y de estado**, donde se mezclan reglas de negocio, validación, orquestación y persistencia, y donde los módulos de alto nivel dependen de una implementación concreta (`localStorage`) en lugar de una abstracción. **LSP no es aplicable de forma estricta** (no hay herencia de clases); se evalúa como sustituibilidad de contratos y arroja un resultado favorable.

- **Conclusión global del sistema:** diseño **sólido en estructura, parcial en acoplamiento**.
- **Nivel general de calidad del diseño: Medio**.
- **Prioridad de intervención: Alta** (las mejoras impactan directamente la implementación del MVP de esta semana y reducen deuda técnica futura).

---

5. Análisis de Principios SOLID

5.1 S — Single Responsability Principle (SRP)

Definición. Una clase o módulo debe tener una sola razón para cambiar.

Elemento evaluado	Evaluación
`loans.service.ts` (validación + reglas + orquestación de estado + persistencia)	**No cumple**
`resources.service.ts` (CRUD + máquina de estados + reglas de baja + persistencia)	**No cumple**
`auth.service.ts` (autenticación + hashing + expiración + persistencia)	**Cumple parcialmente**
`AppDataContext.tsx` (4 dominios + seeds + persistencia + manejo de error)	**No cumple**
`AuthContext.tsx` (sesión + acciones de auth)	**Cumple parcialmente**
`validators.ts` / `dateUtils.ts`	**Cumple**
`guards` + `StatusBadge`	**Cumple**

Evidencia observada:

- `createLoan`, `registerDelivery`, `registerReturn` mezclan validación de negocio, verificación de solapamiento, cálculo de retraso, cambio de estado del recurso y persistencia (`design.md:406-413`, `design.md:417-420`).
- `updateResourceStatus` valida transiciones, cambia el estado y **además** escribe el historial (`design.md:403`, `design.md:681-691`).
- `AppDataContext` reúne `resources`, `loans`, `users` y `sanctions` en un solo reducer, además de inicialización (`seedData`) y sincronización (`design.md:92-98`, `design.md:287-314`).
- `validators.ts` exporta funciones unitarias y puras (cada una con una razón de cambio) — buen contraste (`tasks.md:32-41`).
- `PrivateRoute` y `RoleGuard` tienen una única responsabilidad cada uno (`design.md:355-368`).

Hallazgos:

- Múltiples razones de cambio en C1/C2 (cambios en reglas de negocio *y* en el mecanismo de persistencia fuerzan a tocar el mismo archivo).

- El contenedor de estado C4 cambia por cualquiera de los dominios que agrupa.
- El nombre `loans.service` agrupa responsabilidades de "préstamo" y de "disponibilidad de recurso".

Impacto técnico potencial:

- Regresiones cruzadas: tocar el cálculo de retraso puede afectar la creación de préstamos y la persistencia.
- Dificultad para probar en aislamiento (se necesita `localStorage` para probar reglas puras).
- Mayor superficie de merge conflicts en trabajo paralelo.

Recomendaciones:

- Extraer la **validación de negocio** a funciones puras (reusar `validators` / `dateUtils`) y dejar al servicio la orquestación.
- Extraer el **acceso a datos** a un módulo de persistencia (`storage`) y que el servicio dependa de él.
- Separar en `resources.service` la **máquina de estados** (`resourceStateMachine`) del CRUD.
- Dividir `AppDataContext` por dominio (`ResourcesContext`, `LoansContext`, `UsersContext`) o exponer *selectores* especializados.

5.2 O — Open Closed Principle (OCP)

Definición. Las entidades de software deben estar abiertas para extensión, pero cerradas para modificación.

Elemento evaluado	Evaluación
`loans.service.ts` (`registerReturn` condicional por condición)	**No cumple**
`resources.service.ts` (tabla de transiciones incrustada)	**No cumple**
`auth.service.ts` (flujo de login/registro)	**Cumple parcialmente**
`AppDataContext.tsx` (reducer por `switch` de acciones)	**Cumple parcialmente**
`AuthContext.tsx`	**Cumple parcialmente**
`validators.ts` / `dateUtils.ts` (extensión = nueva función)	**Cumple**
`RoleGuard` (`allowedRoles`) y `StatusBadge` (mapa de estados)	**Cumple**

Evidencia observada:

- `registerReturn` decide el nuevo estado del recurso con una condicional implícita (`Dañado → En_Mantenimiento`, si no `Disponible`); una nueva condición exige modificar la función (`design.md:410`, `design.md:419`, `design.md:669-673`).
- Las transiciones válidas están **enumeradas dentro de la lógica** de `updateResourceStatus`; agregar una transición obliga a editar el módulo (`design.md:403`, `design.md:681-691`).
- Contraste positivo: `RoleGuard` recibe `allowedRoles: UserRole[]`, por lo que **agregar roles no modifica el guard** (`design.md:361-367`); `StatusBadge` representa la extensión como **datos** (mapa estado→color) (`tasks.md:136-139`).
- Los validadores se extienden añadiendo funciones nuevas sin tocar las existentes (`design.md:462`, `tasks.md:32-41`).

Hallazgos:

- Puntos de extensión "rígidos" en la capa de servicios: tablas de decisión y condicionales sobre valores de dominio.
- Riesgo de `if/switch` crecientes conforme se agreguen condiciones/estados (especialmente al llegar los requisitos Post-MVP).

Impacto técnico potencial:

- Cada nueva regla de préstamo/estado implica modificar código estable y re-testear todo el módulo.
- Propagación de cambios a los tests PBT ya escritos.

Recomendaciones:

- Modelar las transiciones como una **tabla declarativa** (`Record<ResourceStatus, ResourceStatus[]>`) o un *state machine* configurable.
- Aplicar el patrón **Strategy** ya previsto en el diseño para las reglas de devolución según `ResourceCondition` (`design.md:462`): un mapa `condición → (nuevoEstado, requiereMantenimiento)`.
- Mantener el reducer de contextos basado en un **mapa de handlers** en lugar de un `switch` largo.

5.3 L — Liscov Substitution Principle (LSP)

Definición. Los objetos de una subclase deben poder sustituir a los de su clase base sin alterar el comportamiento esperado del sistema.

> **Aclaración metodológica:** el proyecto es de naturaleza **funcional** (React + funciones/hooks), **sin jerarquías de herencia**. Por tanto LSP no se evalúa como herencia de clases, sino como **sustituibilidad de contratos**: qué ocurre si un componente se reemplaza por otra implementación que cumple el mismo contrato.

Elemento evaluado	Evaluación
Contrato `OperationResult<T>` en todos los servicios	**Cumple**
Uniones discriminadas `ResourceStatus` / `LoanStatus` / `ResourceCondition`	**Cumple**
Funciones "Strategy" de `validators` (intercambiables)	**Cumple**
Sustitución futura `mockDb` → API real	**Cumple (con condición)**
Herencia de clases	**No aplica**

Evidencia observada:

- Todos los servicios retornan `OperationResult<T>` y **no lanzan excepciones**, por lo que cualquier implementación del contrato es sustituible sin cambiar al cliente (`design.md:245-249`, `design.md:541`).
- Los estados se modelan como **uniones de literales** y los clientes hacen *switching* exhaustivo, sin depender de subtipos concretos (`design.md:152-193`; `tasks.md:136-139`).
- Las funciones de validación comparten firma homogénea (`s → boolean`), lo que permite sustituirlas (patrón Strategy) sin romper el contrato (`design.md:462`).

Hallazgos:

- Fortaleza: contratos estables y uniformes que favorecen la sustitución `mockDb` → backend real.
- Riesgo latente: si una implementación futura **lanzara** excepciones o devolviera un `OperationResult` parcialmente relleno, rompería la sustituibilidad. La recomendación es reforzar el contrato con **postcondiciones** documentadas/validadas.

Impacto técnico potencial:

- Bajo hoy; alto a futuro si se introduce una API real sin respetar el contrato.
- Facilita el testing con *fakes/stubs* que implementen el mismo contrato.

Recomendaciones:

- Documentar y (si es posible) **testear** las postcondiciones de `OperationResult` (nunca lanzar; siempre `{ success, data | error }`).
- Definir **interfaces de servicio** (`LoansService`, `ResourcesService`, `AuthService`) que tanto el mock como la futura API real implementen.

5.4 I — Interface Segregation Principle (ISP)

Definición. Los clientes no deben depender de interfaces que no utilizan.

Elemento evaluado	Evaluación
`useAppData()` (expone recursos + préstamos + usuarios + sanciones)	**No cumple**
`useAuth()` (sesión + login + logout + register)	**Cumple parcialmente**
Props de componentes (`ResourceCard`, `StatusBadge`, `Toast`)	**Cumple**
Módulos de servicio de dominio (API cohesionada por dominio)	**Cumple**

Evidencia observada:

- `useAppData()` entrega **todo el dominio a la vez**; un componente que solo necesita `resources` depende también de `loans`, `users` y `sanctions` (`design.md:92-98`, `design.md:459`).
- `useAuth()` agrupa estado de sesión y acciones (`design.md:459`); aceptable, pero mezcla lectura y mutación en un solo contrato.
- Contraste positivo: las interfaces de props son **estrechas y específicas** (`ResourceCard` recibe `resource` y un callback opcional; `StatusBadge` solo `status`) (`design.md:119-128`).

Hallazgos:

- Interfaz de contexto "ancha": los consumidores reciben más superficie de la que usan, acoplando su (re)renderizado a cambios irrelevantes.

Impacto técnico potencial:

- Re-renders innecesarios y acoplamiento entre features no relacionadas.
- Mayor dificultad para razonar qué consume cada componente.

Recomendaciones:

- Exponer **hooks por dominio** (`useResources`, `useLoans`, `useUsers`) o *selectores* que devuelvan solo el slice necesario.

- Separar en `useAuth` la lectura de sesión (`useSession`) de las acciones (`useAuthActions`).

5.5 D — Dependency Inversion Principle (DIP)

Definición. Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

Elemento evaluado	Evaluación
`loans.service.ts` / `resources.service.ts` / `auth.service.ts` (uso directo de `localStorage`)	**No cumple**
`AppDataContext.tsx` (depende del módulo concreto `mockDb`)	**No cumple**
`AuthContext.tsx` (delega en `auth.service`)	**Cumple parcialmente**
`utils` (puras, sin dependencias)	**Cumple**
Guardias (dependen de la abstracción `useAuth`)	**Cumple parcialmente**

Evidencia observada:

- El diseño **declara** el patrón *Repository* en `mockDb` (`design.md:463`), pero los módulos de alto nivel importan directamente el módulo concreto y este importa `localStorage` (`design.md:374`, `design.md:378-420`).
- Los contextos consumen las funciones concretas del servicio (`design.md:92-98`), sin una interfaz/`port` intermedia.
- Contraste positivo: los **custom hooks** actúan como *fachadas* (`design.md:459`, `design.md:464-465`) y las guardias dependen de `useAuth()`, es decir, de una abstracción, no de `localStorage`.
- La mención de que las funciones puras no dependen de `localStorage` (`tasks.md:454`) confirma que la lógica validable podría aislarse fácilmente.

Hallazgos:

- Inversión de dependencias **incompleta**: existe el patrón en el discurso de diseño, pero la dirección real de las dependencias sigue siendo "alto → bajo concreto".
- `localStorage` (detalle de infraestructura) "sangra" hacia servicios y contextos, dificultando el testing y el futuro reemplazo por una API.

Impacto técnico potencial:

- Tests acoplados al entorno del navegador; difícil inyectar *fakes* de almacenamiento.
- Migrar a backend real implicaría tocar servicios y contextos en varios puntos.

Recomendaciones:

- Definir un **puerto** `StorageRepository` (interfaz) y una implementación `LocalStorageRepository`; que los servicios dependan del puerto.
- Definir **interfaces de servicio** y que los contextos dependan de ellas (inyección o *factory*), facilitando mocks.
- Aislar `hashUtils` / `uuidUtils` tras una abstracción para poder sustituirlos.

6. Hallazgos transversales

Problemas recurrentes identificados:

1. **Concentración de responsabilidades en servicios y contextos (SRP):** los módulos de negocio mezclan validación, reglas, orquestación de estado y persistencia; los contextos agrupan múltiples dominios.
2. **Puntos de extensión rígidos (OCP):** tablas de decisión y condicionales incrustados (transiciones de estado y reglas de devolución) que obligan a modificar código estable.
3. **Inversión de dependencias incompleta (DIP):** `localStorage` y los módulos concretos son dependencias directas de la lógica de alto nivel; el patrón Repository existe solo a nivel declarativo.
4. **Interfaz de estado ancha (ISP):** `useAppData()` expone todos los dominios juntos, acoplando a los consumidores a datos que no usan.
5. **LSP no aplica estrictamente**, pero el contrato `OperationResult<T>` es una fortaleza que ya habilita sustituibilidad (mock → API real) y testing con fakes.
6. **Patrón positivo a replicar:** utilidades puras y guardias declarativas (`allowedRoles`, mapa de estados) ya cumplen SRP/OCP con alto nivel.

7. Priorización de mejoras

#	Hallazgo	Principio	Severidad	Esfuerzo	Prioridad	Acción sugerida
M1	Servicios mezclan reglas de negocio, orquestación de estado y persistencia	SRP	**Alta**	Medio	**Alta**	Extraer validación pura (reusar `validators` / `dateUtils`), separar persistencia en `storage` y aislar la máquina de estados.
M2	Dependencia directa de `localStorage` en servicios y contextos	DIP	**Alta**	Medio	**Alta**	Introducir puerto `StorageRepository` + implementación `LocalStorageRepository`; definir interfaces de servicio e inyectarlas.
M3	Tabla de transiciones de estado incrustada en la lógica	OCP	**Alta**	Bajo	**Alta**	Reemplazar por una **tabla declarativa** `Record<ResourceStatus, ResourceStatus[]>` o *state machine* configurable.
M4	`AppDataContext` agrupa 4 dominios y expone todo con `useAppData`	SRP + ISP	**Media**	Medio	**Alta**	Dividir en contextos/hooks por dominio o exponer selectores (`useResources` , `useLoans` , `useUsers`).
M5	Condicional por `ResourceCondition` en `registerReturn`	OCP	**Media**	Bajo	**Media**	Aplicar **Strategy** con mapa `condición → (nuevoEstado, requiereMantenimiento)` .
M6	`auth.service` / `AuthContext` concentran hashing, expiración y sesión	SRP	**Media**	Bajo	**Media**	Extraer `hashUtils` / `uuidUtils` (ya previstos) y una política de sesión desacoplada.
M7	Reducer de contextos basado en `switch` largo	OCP	**Baja**	Bajo	**Baja**	Mapa de handlers por tipo de acción.
M8	Postcondiciones de `OperationResult` no reforzadas	LSP	**Baja**	Bajo	**Media**	Documentar/testear invariantes (nunca lanzar; `{ success, data | error }`).

Recomendación de secuencia: aplicar **M3** → **M1** → **M2** durante las tareas 3–5 de `tasks.md` (utilidades → servicios → contextos), y **M4** al construir los hooks de feature. **M5** y **M7** son refinamientos de bajo esfuerzo que pueden hacerse en la misma pasada.

---

9. Conclusión

Síntesis del estado del diseño: arquitectura **bien estratificada** y coherente con el dominio, con capas claras (Pages → Features → Infraestructura → Shared) y artefactos puros/declarativos correctos (`utils`, `guards`). La deuda se localiza en la **capa de servicios y de estado**, donde se concentran responsabilidades y no se ha concretado la inversión de dependencias.

Principios más comprometidos: SRP, **DIP** y **OCP** en la capa de servicios/estado.

Principios mejor implementados: OCP e ISP en utilidades y componentes de vista, y **LSP** (como sustituibilidad de contratos con `OperationResult<T>`), aunque su evaluación estricta no aplique por ser un diseño funcional.

Nivel de deuda técnica asociada al diseño: Media, mitigable antes de escribir el código del MVP.

Recomendación final: implementar el MVP incorporando desde el inicio un **puerto de persistencia** (`StorageRepository`), la **máquina de estados declarativa** y la **segregación de contextos/hooks por dominio**. Esto preserva las fortalezas actuales, elimina los riesgos de acoplamiento y deja el sistema preparado para (a) migrar de `localStorage` a un backend real y (b) escalar la lógica Post-MVP sin reescribir los módulos estables.

Versión corta para revisión rápida

Principio	¿Se cumple?	Evidencia	Problema detectado	Recomendación
SRP	**Parcial**	`loans.service` / `resources.service` mezclan validación + orquestación + persistencia; `AppDataContext` agrupa 4 dominios. `utils` / `guards` cumplen.	Múltiples razones de cambio por módulo.	Separar validación (pura), persistencia y máquina de estados; dividir contextos por dominio.
OCP	**Parcial**	Transiciones de estado y condición de devolución incrustadas. `RoleGuard` / `validators` extensibles.	`if/switch` y tablas incrustadas obligan a modificar código estable.	Tabla declarativa de transiciones y patrón Strategy por condición.
LSP	**Sí (N/A estricto)**	Contrato uniforme `OperationResult<T>`; uniones discriminadas; validadores intercambiables.	Riesgo futuro si una implementación rompe el contrato.	Documentar/testear postcondiciones; interfaces de servicio sustituibles.
ISP	**Parcial**	`useAppData()` expone todos los dominios; props de componentes correctas.	Consumidores dependen de datos que no usan.	Hooks/selectores por dominio; separar lectura y acciones en auth.
DIP	**Parcial**	Servicios/contextos importan `localStorage` y módulos concretos; hooks actúan como fachada.	Lógica de alto nivel acoplada a infraestructura.	Puerto `StorageRepository` + interfaces de servicio inyectadas.